

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ

УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «НИЖЕГОРОДСКИЙ
ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ ИМ. Н.И. ЛОБАЧЕВСКОГО»
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИНСТИТУТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ, МАТЕМАТИКИ И
МЕХАНИКИ

**КАФЕДРА ВЫСОКОПРОИЗВОДИТЕЛЬНЫХ ВЫЧИСЛЕНИЙ И
СИСТЕМНОГО ПРОГРАММИРОВАНИЯ**

**ОТВЕТЫ НА ВОПРОСЫ
ПО ДИСЦИПЛИНЕ
«ОБУЧЕНИЕ С ПОДКРЕПЛЕНИЕМ»**

Выполнил: Рштуни Владислав

Группа: 3825М1сФИ1

Нижний Новгород

2026

Содержание

Вопросы из недели 1	2
1. Базовые понятия RL	2
2. Метод кросс-энтропии (табличный + нейронная сеть)	2
Вопросы из недели 2	4
1. $V(s)$, $Q(s, a)$ - что это такое	4
2. Метод Policy iteration	5
Вопросы из недели 3	6
1. Q-learning, SARSA	6
2. Exploration vs Exploitation	6
Вопросы из недели 4	8
1. DQN	8
2. Experience replay	8
3. Target network	9
4. Double DQN	9
Вопросы из недели 6	11
1. Policy gradient	11
2. REINFORCE	11
3. Actor-Critic	12
Вопросы из недели 7	13
1. Применение RNN для генерации последовательностей	13
2. Применение RL к дообучению моделей генерации последовательностей	13

Вопросы из недели 1

1. Базовые понятия RL

Обучение с подкреплением (Reinforcement Learning, RL) — это парадигма машинного обучения, в которой агент обучается оптимальной стратегии взаимодействия со средой методом проб и ошибок. Главное отличие от обучения с учителем заключается в отсутствии заранее размеченных эталонных ответов: агент самостоятельно генерирует данные, выбирая действия, и получает за них числовую обратную связь (вознаграждение или штраф). При этом агент напрямую влияет на распределение своих будущих наблюдений.

Взаимодействие формально описывается Марковским процессом принятия решений (MDP), который включает:

- множество состояний $s \in S$,
- множество действий $a \in A$,
- функцию вознаграждения $r \in R$,
- динамику среды $P(s_{t+1}|s_t, a_t)$, задающую вероятность перехода.

Ключевым является марковское свойство: будущее состояние зависит только от текущего состояния и выбранного действия, а не от всей предыдущей истории: $P(s_{t+1}|s_t, a_t, s_{t-1}, \dots) = P(s_{t+1}|s_t, a_t)$

Стратегия агента задается политикой $\pi(a|s)$, определяющей вероятность выбора действия a в состоянии s . Цель RL-алгоритма - найти такую политику, которая максимизирует математическое ожидание суммарного дисконтированного вознаграждения за эпизод: $E_{\pi}[\sum_{t=0}^T \gamma^t r_t]$

где $\gamma \in (0, 1]$ - коэффициент дисконтирования, обеспечивающий сходимость бесконечных сумм.

2. Метод кросс-энтропии (табличный + нейронная сеть)

Метод кросс-энтропии (Cross-Entropy Method, CEM) — это

эволюционный подход, работающий по циклу:

1. сыграть,
2. отобрать лучшие,
3. обновить политику.

На каждой итерации агент генерирует N полных эпизодов, вычисляет для каждого суммарное вознаграждение и выбирает M лучших траекторий, называемых элитными сессиями (elite sessions). Политика затем перестраивается так, чтобы с большей вероятностью воспроизводить действия, встречавшиеся в этих элитных траекториях.

Табличный вариант применяется при небольших конечных пространствах состояний. Политика хранится в виде матрицы вероятностей $\pi(a|s) = A_{s,a}$. Обновление сводится к частотному оцениванию: для каждого состояния s вероятность действия a вычисляется как доля раз, когда агент выбирал a в s среди всех посещений s в элитных сессиях.

Формула:

$$\pi(a|s) = \frac{\sum_{(s_t, a_t) \in Elite} [s_t = s][a_t = a]}{\sum_{(s_t, a_t) \in Elite} [s_t = s]}$$

Для задач с большими или непрерывными пространствами используется аппроксимированный СЕМ. Политика параметризуется моделью (чаще всего нейронной сетью) $\pi_W(a|s)$. Веса сети обновляются путем максимизации логарифмического правдоподобия действий из элиты:

$$W_{i+1} = W_i + \alpha \nabla \sum_{(s_i, a_i) \in Elite} \log \pi_{W_i}(a_i|s_i).$$

На практике это реализуется как обычное обучение с учителем, где состояния выступают признаками, а действия из элиты - целевыми метками. Для непрерывных пространств действий политика часто моделируется гауссовским распределением $N(\mu(s), \sigma^2)$, где среднее $\mu(s)$ предсказывается нейросетью.

Вопросы из недели 2

1. $V(s)$, $Q(s, a)$ - что это такое

В RL цели агента формализуются через максимизацию суммарного дисконтированного вознаграждения (return): $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$.

Для оценки качества поведения вводятся две фундаментальные функции ценности.

- Функция ценности состояния $V^{\pi}(s)$ (State-value function) — это математическое ожидание возврата при условии, что агент начинает в состоянии s и далее следует стратегии π . Интуитивно $V(s)$ отвечает на вопрос: насколько выгодно находиться в данном состоянии, если продолжать играть согласно текущей политике. При вычислении $V(s)$ учитывается стохастичность как самой политики $\pi(a|s)$, так и динамики среды.
- Функция ценности действия $Q^{\pi}(s, a)$ (Action-value function) — это ожидаемый возврат при условии, что агент находится в состоянии s , принудительно выбирает конкретное действие a , а все последующие шаги выполняет согласно политике π . Ключевое отличие от $V(s)$ заключается в том, что на первом шаге стохастичность политики отключается. Интуиция: какой выигрыш принесет конкретный шаг a в состоянии s при последующем следовании π .

Они связаны уравнениями ожидания Беллмана:

$$Q^{\pi}(s, a) = E[r + \gamma V^{\pi}(s') | s, a]$$

и

$$V^{\pi}(s) = \sum_a \pi(a|s) Q^{\pi}(s, a).$$

2. Метод Policy iteration

Policy iteration (итерация по политикам) - классический алгоритм решения MDP, входящий в общую схему Generalized Policy iteration (GPI).

Алгоритм чередует два этапа, повторяя цикл до достижения оптимальности.

- 1) Оценка политики (Policy Evaluation): для фиксированной политики π вычисляется её функция ценности $V^\pi(s)$. Итеративно применяется уравнение ожидания Беллмана:

$$V_{k+1}(s) \leftarrow \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V_k(s')].$$

Процесс продолжается до сходимости (когда изменение V во всех состояниях становится меньше заданного допуска).

- 2) Улучшение политики (Policy Improvement): получив V^π , алгоритм вычисляет $Q^\pi(s, a)$ и обновляет политику жадным образом: $\pi'(s) = \arg \max_a Q^\pi(s, a)$. Согласно лемме об улучшении политики, такая замена гарантирует, что новая политика π' будет не хуже старой: $V^{\pi'}(s) \geq V^\pi(s)$ для всех s .

Цикл повторяется, пока политика не перестанет меняться ($\pi' = \pi$). Это означает, что политика уже удовлетворяет уравнению оптимальности Беллмана, и найденное решение является оптимальной детерминированной политикой.

Вопросы из недели 3

1. Q-learning, SARSA

Q-learning и SARSA - два базовых model-free алгоритма, основанных на методе временных разностей (Temporal Difference, TD). Оба работают с последовательностями $\langle s, a, r, s' \rangle$ и итеративно обновляют таблицу Q-значений по формуле:

$$Q(s_t, a_t) \leftarrow (1 - \alpha)Q(s_t, a_t) + \alpha \tilde{Q}(s_t, a_t),$$

где α - скорость обучения. Разница заключается в целевом значении $\tilde{Q}$.

Q-learning является off-policy методом. Он вычисляет целевую оценку жадно относительно следующего состояния: $\tilde{Q}(s, a) = r + \gamma \max_{a'} Q(s', a')$.

Алгоритм предполагает, что начиная со следующего состояния агент будет всегда выбирать действие с максимальным Q-значением, независимо от того, какую политику он использует для сбора данных. Это позволяет обучить оптимальную политику даже на данных, сгенерированных случайной стратегией.

SARSA (State-Action-Reward-State-Action) является on-policy методом. Он использует фактическое следующее действие a' , которое агент на самом деле выбрал согласно текущей политике π : $\tilde{Q}(s, a) = r + \gamma Q(s', a')$.

Поскольку обновление напрямую зависит от сгенерированного агентом шага, SARSA оценивает ценность именно той политики, которая управляет агентом в данный момент, включая её исследовательскую составляющую, что делает алгоритм более осторожным в рискованных средах.

2. Exploration vs Exploitation

Дилемма исследования против использования возникает из-за того, что агент не обладает априорным знанием среды.

- Exploitation (использование) — это выбор действия с текущим максимальным Q-значением для получения максимальной награды здесь и сейчас.

- Exploration (исследование) — это выбор случайных или менее изученных действий для сбора новых данных и потенциального обнаружения стратегий с большей суммарной наградой.

Если использовать только exploitation, агент быстро застревает в локальном оптимуме. Для баланса используются вероятностные стратегии:

- $\epsilon - greedy$: с вероятностью ϵ агент выбирает случайное действие, а с вероятностью $1 - \epsilon$ - оптимальное согласно текущей таблице Q. Для гарантии сходимости параметр ϵ плавно уменьшают до нуля со временем.
- Softmax (Болцмановское исследование): вероятность выбора действия пропорциональна экспоненте его нормализованного Q-значения:

$$\pi(a|s) = \frac{\exp(Q(s, a)/\tau)}{\sum_{a'} \exp(Q(s, a')/\tau)}.$$

Параметр температуры τ регулирует степень случайности.

Вопросы из недели 4

1. DQN

DQN (Deep Q-Network) — это глубокое расширение классического Q-learning для работы с большими или непрерывными пространствами состояний, где табличное хранение значений $Q(s, a)$ невозможно. Вместо явной таблицы DQN использует нейронную сеть с параметрами θ для аппроксимации Q-функции: $Q(s, a; \theta)$. На вход сети подается только состояние s , а на выходе за один прямой проход генерируется вектор Q-значений для всех возможных действий.

Обучение формулируется как задача регрессии с минимизацией среднеквадратичной ошибки (MSE) между предсказанным значением и целевым TD-таргетом:

$$L(\theta) = E \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta) - Q(s, a; \theta) \right)^2 \right]$$

Наивное применение нейросетей сталкивается с проблемами корреляции последовательных данных и нестабильности цели, что решается методами Experience Replay и Target Network.

2. Experience replay

Experience replay - механизм буферизации и повторного использования прошлого опыта взаимодействия агента со средой. При стандартном online-обучении последовательные переходы $\langle s, a, r, s' \rangle$ обладают сильной временной автокорреляцией, что нарушает требования стохастического градиентного спуска к независимости выборок (i.i.d.) и приводит к нестабильности обучения.

В методе experience replay все собранные переходы сохраняются в кольцевой буфер (replay buffer) фиксированного размера. На этапе обучения мини-батчи формируются путем случайной равномерной выборки из этого буфера. Это эффективно перемешивает данные, сглаживает градиенты и

позволяет многократно переиспользовать редкие или информативные переходы. Важное ограничение: метод работает исключительно с off-policy алгоритмами (такими как DQN), так как данные в буфере собраны под старой политикой агента.

3. Target network

Target Network (целевая сеть) - архитектурный прием, решающий проблему движущейся цели (moving target). В базовой формуле TD-ошибки целевое значение $r + \gamma \max_{a'} Q(s', a'; \theta)$ вычисляется той же сетью с параметрами θ , которые обновляются на каждом шаге. Из-за этого возникает нестабильная автокорреляция: небольшое изменение весов сразу меняет цель для следующего обновления.

Идея заключается во введении второй копии сети с параметрами θ^- , веса которой фиксируются на протяжении нескольких итераций. Целевое значение теперь вычисляется как $r + \gamma \max_{a'} Q(s', a'; \theta^-)$, что делает его константой на промежутке между обновлениями. Существуют два способа синхронизации:

- 1) Жесткое обновление (Hard update): раз в N шагов веса целевой сети полностью копируются из основной: $\theta^- \leftarrow \theta$.
- 2) Мягкое обновление (Soft update): на каждом шаге применяется экспоненциальное сглаживание: $\theta^- \leftarrow (1 - \tau)\theta^- + \tau\theta$.

4. Double DQN

Double DQN был предложен для устранения систематической ошибки переоценки (overestimation bias), присущей классическому Q-learning. Эта ошибка возникает из-за оператора $\max$ в целевой функции: поскольку Q-значения содержат случайный шум аппроксимации, математическое

ожидание максимума всегда больше или равно максимуму математических ожиданий ($E[\max Q] \geq \max E [Q]$).

Алгоритм решает эту проблему, разделяя этапы выбора действия и оценки его ценности, переиспользуя уже имеющиеся основную и целевую сети DQN.

Действие выбирается основной сетью: $a^* = \arg \max_{a'} Q(s', a'; \theta)$, а его ценность оценивается целевой сетью: $Q(s', a^*; \theta^-)$. Итоговый таргет принимает вид:

$$y = r + \gamma Q(s', \arg \max_{a'} Q(s', a'; \theta); \theta^-).$$

Такое разделение значительно снижает положительное смещение оценок и делает кривую обучения более гладкой.

Вопросы из недели 6

1. Policy gradient

Policy gradient - семейство методов, которые оптимизируют стохастическую политику $\pi_\theta(a|s)$ напрямую, без промежуточного этапа оценки функции ценности. В отличие от value-based подходов, метод максимизирует математическое ожидание суммарного вознаграждения $J(\theta) = E[R]$ непосредственно через градиентный подъем по параметрам θ .

Ключевая математическая идея - логарифмический производный трюк (log-derivative trick):

$$\nabla \pi_\theta(a|s) = \pi_\theta(a|s) \nabla \log \pi_\theta(a|s)$$

Подстановка этого соотношения преобразует градиент целевой функции в математическое ожидание:

$$\nabla J(\theta) = E_{s \sim d^\pi, a \sim \pi_\theta} [\nabla \log \pi_\theta(a|s) \cdot Q(s, a)]$$

Это означает увеличение вероятности действий, приводящих к высоким возвратам, и уменьшение вероятности неудачных действий, масштабируя обновление логарифмической производной политики.

2. REINFORCE

REINFORCE - базовый алгоритм Policy Gradient, использующий метод Монте-Карло для оценки градиента. Алгоритм работает строго on-policy и требует полных эпизодов взаимодействия. На каждой итерации агент генерирует N траекторий, для каждого шага вычисляет полный дисконтированный возврат $G_t = \sum_{k=t}^T \gamma^{k-t} r_k$, который служит несмещенной оценкой $Q(s_t, a_t)$. Градиент оценивается как

$$\nabla J \approx \frac{1}{N} \sum_{i=1}^N \sum_t \nabla \log \pi_\theta(a_t|s_t) \cdot G_t.$$

Главная проблема REINFORCE - высокая дисперсия оценок из-за накопления шума среды по всему эпизоду. Дисперсию можно радикально снизить без смещения градиента с помощью базисной функции (baseline)

$b(s)$. Добавление вычитаемого слагаемого $b(s)$ не меняет математическое ожидание градиента, так как $E[\nabla \log \pi_\theta(a|s)b(s)] = 0$, но уменьшает разброс значений, если $b(s)$ коррелирует с $Q(s, a)$.

3. Actor-Critic

Actor-Critic - гибридный подход, объединяющий прямую оптимизацию политики (Actor) и обучение функции ценности (Critic) для снижения дисперсии градиентов и ускорения сходимости. Архитектура состоит из двух нейронных сетей: Actor $\pi_\theta(a|s)$ отвечает за выбор действий и обновляется через Policy Gradient, а Critic $V_\phi(s)$ обучается предсказывать ожидаемый возврат из состояния и служит baseline для Actor.

Вместо использования полных возвратов G_t , Actor оптимизируется по функции преимущества (Advantage):

$$A(s, a) = Q(s, a) - V(s),$$

которая оценивается через одношаговую TD-ошибку:

$$A(s, a) \approx r + \gamma V(s') - V(s).$$

Это показывает, насколько выбранное действие лучше среднего для данного состояния. Обновление происходит параллельно:

- Critic минимизирует MSE-ошибку предсказания:

$$L_{critic} = (V(s) - [r + \gamma V(s')])^2,$$

- Actor делает шаг в направлении:

$$\nabla J_{actor} \approx \sum \{\nabla \log \pi_\theta\}$$

Вопросы из недели 7

1. Применение RNN для генерации последовательностей

Для задач генерации последовательностей (машинный перевод, диалоговые системы) классически используется архитектура Encoder-Decoder на базе RNN или трансформеров. Энкодер считывает входные данные и формирует скрытое состояние, а декодер пошагово генерирует выходную последовательность.

При стандартном обучении с учителем декодер предсказывает следующий токен на основе эталонной (reference) последовательности, минимизируя кросс-энтропию. Однако существует фундаментальное различие между режимами обучения и инференса. Во время инференса модель работает в режиме сэмплирования или жадного выбора, подавая на вход свои собственные предыдущие предсказания.

Это порождает критическую проблему сдвига распределения (distribution shift или exposure bias): распределение последовательностей, генерируемых моделью во время инференса, отличается от распределения эталонных данных обучения. Если модель допустит ошибку, её скрытые состояния смещаются, что приводит к лавинообразному накоплению ошибок. Кроме того, наличие нескольких корректных вариантов ответа заставляет модель усреднять вероятности, выдавая шаблонные фразы.

2. Применение RL к дообучению моделей генерации последовательностей

Применение RL позволяет решить проблемы supervised-подхода, так как модель обучается на собственных сэмплированных действиях, что устраняет distribution shift. Задача формулируется как максимизация ожидаемой награды $J = E[R(s, a)]$ с помощью policy gradient:

$$\nabla J = [\nabla \log \pi_{\theta}(a|s) \times Q(s, a)]$$

Для снижения высокой дисперсии оценок в генерации последовательностей применяется метод Self-Critical Sequence Training

(SCST). В качестве baseline используется награда от greedy-инференса (жадного вывода) той же самой модели в реальном времени:

$$A(s, a) = R(s, a_{\text{sampled}}) - R(s, a_{\text{greedy}})$$

Это позволяет сравнивать сэмплированную последовательность с детерминированной, значительно уменьшая шум градиента без необходимости обучать отдельную сеть-критик.

В контексте современных больших языковых моделей (LLM) применяется RLHF (Reinforcement Learning from Human Feedback). Пайплайн включает:

- 1) Supervised Fine-Tuning (SFT) на качественных парах промпт-ответ;
- 2) обучение Reward Model на парных сравнениях ответов человеком;
- 3) дообучение политики через алгоритм PPO с наградой от Reward Model и штрафом (KL-дивергенция) за отклонение от исходной SFT-модели.

Альтернативой является Direct Preference Optimization (DPO), который упрощает процесс, оптимизируя логарифмическое правдоподобие предпочтений напрямую, без явного обучения reward model и использования PPO.